



GRAFOS PONDERADOS

ESTRUTURA DE DADOS

CST em Desenvolvimento de Software Multiplataforma



PROF. Me. TIAGO A. SILVA



LIVRO DE REFERÊNCIA DA DISCIPLINA

- **BIBLIOGRAFIA BÁSICA:**

- GRONER, Loiane. **Estrutura de dados e algoritmos com JavaScript**: escreva um código JavaScript complexo e eficaz usando a mais recente ECMAScript. **São Paulo: Novatec Editora, 2019.**

- **NESTA AULA:**

- Capítulo 12 – Grafos



PARA SOBREVIVER AO JAVASCRIPT

Non-zero value



null



0



undefined

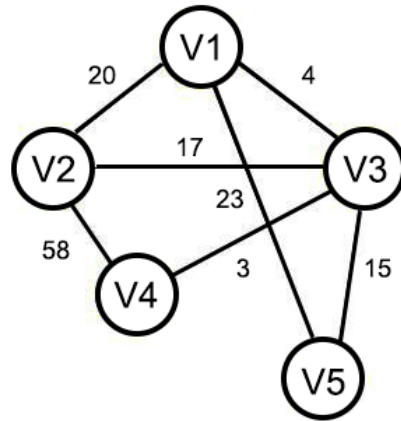


O QUE SÃO GRAFOS PONDERADOS?

- Um grafo ponderado (ou grafo com pesos) é um tipo de grafo em que cada aresta possui um valor associado, chamado de peso ou custo.
- Esse peso pode representar:
 - Distância entre cidades
 - Tempo de viagem
 - Custo de envio
 - Largura de banda
 - Qualquer outra medida associada à conexão entre dois nós
- Em um grafo ponderado, cada aresta (ligação entre dois vértices) tem uma informação extra: **o peso**.
- Se o grafo for direcionado, o peso só vale na direção da aresta.

O QUE SÃO GRAFOS PONDERADOS?

Matriz de Adjacência grafo ponderado



*zero é um valor escolhido em código para considerar não ter nenhuma ligação entre os dois grafos, porém se seu grafos tiver zero como um valor valido deve se escolher outro valor.

	V1	V2	V3	V4	V5
V1	0*	20	4	0*	23
V2	20	0*	17	58	0*
V3	4	17	0*	3	15
V4	0*	58	3	0*	0*
V5	23	0*	15	0*	0*

Imagem: Paulo Martins

MÉTODO CONSTRUTOR

```
1  class GrafoPonderado {  
2  
3      constructor() {  
4          // Conjunto de vértices únicos  
5          this.vertices = new Set();  
6  
7          // Mapa onde cada vértice aponta para uma lista de objetos: { vertice, peso }  
8          this.adjacencia = new Map();  
9      }
```

ADICIONAR VÉRTICES

```
10
11 // Adiciona um novo vértice ao grafo. Se já existir, nada é feito.
12 // Também inicializa sua lista de adjacência.
13 adicionarVertice(v) {
14
15     // Garante que o vértice está no conjunto
16     this.vertices.add(v);
17     if (!this.adjacencia.has(v)) {
18
19         // Inicializa lista de vizinhos se ainda não existir
20         this.adjacencia.set(v, []);
21     }
22 }
```

ADICIONAR ARESTAS

```
23
24 // Adiciona uma aresta ponderada entre dois vértices.
25 // Cria os vértices caso ainda não existam. Por padrão,
26 // é um grafo direcionado.
27 adicionarAresta(origem, destino, peso) {
28     if (!this.adjacencia.has(origem)) this.adicionarVertice(origem);
29     if (!this.adjacencia.has(destino)) this.adicionarVertice(destino);
30
31     this.adjacencia.get(origem).push({ vertice: destino, peso });
32
33     // Se o grafo for não-direcionado, descomente a linha abaixo:
34     // this.adjacencia.get(destino).push({ vertice: origem, peso });
35 }
```


IMPRIMIR O GRAFO - VISUALIZAÇÃO

```
37 // Mostra a representação do grafo como lista de adjacência,  
38 // com os pesos visíveis.  
39 imprimirGrafo() {  
40     for (const [v, vizinhos] of this.adjacencia.entries()) {  
41         const lista = vizinhos.map(obj => `${obj.vertice}(${obj.peso})`).join(', ');  
42         console.log(`${v} -> ${lista}`);  
43     }  
44 }
```

IMPLEMENTAÇÃO DA MATRIZ DE ADJACÊNCIA

```
46 // Gera e imprime a matriz de adjacência do grafo.
47 // Usa Infinity para representar ausência de aresta.
48 imprimirMatrizAdjacencia() {
49     const vertices = Array.from(this.vertices);
50     const n = vertices.length;
51     const matriz = Array.from({ length: n }, () => Array(n).fill(Infinity));
52
53     vertices.forEach((v, i) => {
54         matriz[i][i] = 0; // distância para si mesmo = 0
55         for (const vizinho of this.adjacencia.get(v)) {
56             const j = vertices.indexOf(vizinho.vertice);
57             matriz[i][j] = vizinho.peso;
58         }
59     });
60
61     console.log('Matriz de Adjacência (valores ∞ representam ausência de aresta:');
62     console.log(' ', vertices.join(' '));
63     matriz.forEach((linha, i) => {
64         console.log(vertices[i], linha.map(x => x === Infinity ? '∞' : x).join(' '));
65     });
66 }
```

O QUE É MATRIZ DE ADJACÊNCIA?

- A matriz de adjacência é uma matriz (tabela) bidimensional usada para representar as conexões entre os vértices (nós) de um grafo.
- Se houver uma aresta entre o vértice **i** e o vértice **j**, então a posição **matriz[i][j]** recebe o valor **1** (ou o peso, se for um grafo ponderado).
- Se não houver aresta, o valor é **0** (ou **Infinity** ou **null**, dependendo do caso).



BUSCA EM PROFUNDIDADE – DFS (DEPTH-FIRST SEARCH)

```
68 // Busca em Profundidade (Depth-First Search)
69 // Realiza uma busca em profundidade, visitando
70 // o vértice inicial e seus vizinhos recursivamente
71 // até esgotar os caminhos.
72 dfs(inicio) {
73     const visitados = new Set();
74     const resultado = [];
75
76     const visitar = (v) => {
77         visitados.add(v);
78         resultado.push(v);
79
80         for (const vizinho of this.adjacencia.get(v)) {
81             if (!visitados.has(vizinho.vertice)) {
82                 visitar(vizinho.vertice);
83             }
84         }
85     };
86
87     visitar(inicio);
88     console.log('DFS:', resultado.join(' -> '));
89 }
```



O QUE É BUSCA EM PROFUNDIDADE?

- A Busca em Profundidade percorre um grafo indo o mais fundo possível em cada caminho antes de voltar e explorar outras possibilidades. Ela é feita de forma recursiva ou com o uso de uma pilha.
- É útil para:
 - Explorar todos os vértices alcançáveis de um ponto.
 - Detectar ciclos.
 - Determinar componentes conexos.
 - Resolver labirintos.
 - Construir árvores de espalhamento.



BUSCA EM LARGURA – BFS (BREADTH-FIRST SEARCH)

```
91 // Busca em Largura (Breadth-First Search)
92 // Realiza uma busca em largura, explorando
93 // primeiro os vizinhos mais próximos, usando uma fila.
94 bfs(inicio) {
95     const visitados = new Set();
96     const fila = [inicio];
97     const resultado = [];
98
99     visitados.add(inicio);
100
101     while (fila.length > 0) {
102         const atual = fila.shift();
103         resultado.push(atual);
104
105         for (const vizinho of this.adjacencia.get(atual))
106             if (!visitados.has(vizinho.vertice)) {
107                 visitados.add(vizinho.vertice);
108                 fila.push(vizinho.vertice);
109             }
110     }
111
112     console.log('BFS:', resultado.join(' -> '));
113 }
114 }
```

O QUE É BUSCA EM LARGURA?

- É um algoritmo de exploração de grafos que visita os vértices em camadas, ou seja, primeiro visita todos os vizinhos do vértice inicial, depois os vizinhos desses vizinhos, e assim por diante.
- É ideal para:
 - Descobrir o menor número de passos até um nó (em grafos não ponderados).
 - Explorar todos os nós acessíveis a partir de um ponto.
 - Verificar conectividade.
- Conceitos fundamentais
 - Fila (queue): usada para armazenar os próximos vértices a serem visitados, na ordem em que foram descobertos.
 - Visitado: estrutura (normalmente um objeto ou vetor booleano) usada para evitar visitar o mesmo vértice duas vezes.

COMPARAÇÃO DFS E BFS

	BFS (BUSCA EM LARGURA)	DFS (BUSCA EM PROFUNDIDADE)
Estrutura usada:	Fila	Pilha (ou recursão)
Visita:	Camada por camada	Vai até o fundo e volta
Útil para:	Menor caminho (sem peso)	Detectar ciclos, labirintos
Ordem de visita:	Mais ampla	Mais profunda

IMPLEMENTAÇÃO DO ALGORITMO DE DIJKSTRA

```
116 // Algoritmo de Dijkstra para encontrar o caminho mais curto
117 // Calcula as menores distâncias entre o vértice inicial e
118 // todos os demais, com base nos pesos das arestas.
119 // Usa a abordagem clássica de Dijkstra.
120 dijkstra(inicio) {
121     const distancias = {};
122     const anteriores = {};
123     const naoVisitados = new Set(this.vertices);
124
125     for (const v of this.vertices) {
126         distancias[v] = Infinity;
127         anteriores[v] = null;
128     }
129     distancias[inicio] = 0;
130
131     while (naoVisitados.size > 0) {
132         // Encontra o vértice não visitado com a menor distância conhecida
133         const atual = [...naoVisitados].reduce((a, b) =>
134             distancias[a] < distancias[b] ? a : b
135         );
136         naoVisitados.delete(atual);
137
138         // Atualiza distâncias para vizinhos
139         for (const vizinho of this.adjacencia.get(atual)) {
140             const alt = distancias[atual] + vizinho.peso;
141             if (alt < distancias[vizinho.vertice]) {
142                 distancias[vizinho.vertice] = alt;
143                 anteriores[vizinho.vertice] = atual;
144             }
145         }
146     }
```



IMPLEMENTAÇÃO DO ALGORITMO DE DIJKSTRA - FINALIZAÇÃO

```
147
148     // Exibe o resultado
149     console.log(`Menores distâncias a partir de ${inicio}:`);
150     for (const v of this.vertices) {
151         console.log(`${v}: ${distancias[v]}`);
152     }
153 } // Fecha dijkstra
154 } // Fecha GrafoPonderado
155
156 // Exporta a classe para uso em outros módulos
157 module.exports = GrafoPonderado;
```

EXEMPLO DE USO DA CLASSE

*Importando GrafoPonderado e usando os
métodos*

EXEMPLO DE USO DA CLASSE

```
1  const GrafoPonderado = require('./GrafoPonderado.js');
2
3  // Exemplo de uso:
4  const grafo = new GrafoPonderado();
5  grafo.adicionarAresta('A', 'B', 2);
6  grafo.adicionarAresta('A', 'C', 5);
7  grafo.adicionarAresta('B', 'C', 1);
8  grafo.adicionarAresta('B', 'D', 4);
9  grafo.adicionarAresta('C', 'D', 2);
10
11  grafo.imprimirGrafo();
12  grafo.imprimirMatrizAdjacencia();
13  grafo.dfs('A');
14  grafo.bfs('A');
15  grafo.dijkstra('A');
```



EXERCÍCIOS

Use os métodos da classe GrafoPonderado

EXERCÍCIO 1

- Na Cidade dos Gnomos, as ruas conectam casas mágicas com diferentes distâncias encantadas (pesos).
 - Crie o grafo a seguir e:
 - Imprima a lista de adjacência.
 - Imprima a matriz de adjacência.
 - Use DFS e BFS a partir da Casa A.
 - Use Dijkstra a partir da Casa A para saber o caminho mais rápido até a Casa E.
- Ruas mágicas (arestas):
 - $A \rightarrow B$ (3)
 - $A \rightarrow C$ (2)
 - $B \rightarrow D$ (4)
 - $C \rightarrow D$ (1)
 - $D \rightarrow E$ (5)

EXERCÍCIO 2

- Um trem precisa cruzar uma rede ferroviária entre cidades com diferentes tempos de viagem. Construa o grafo a seguir e responda:
 - Qual a menor distância de São Paulo até Porto Alegre?
 - Mostre os percursos em DFS e BFS a partir de São Paulo.
- Conexões ferroviárias:
 - São Paulo → Campinas (1)
 - Campinas → Curitiba (4)
 - São Paulo → Curitiba (2)
 - Curitiba → Florianópolis (3)
 - Florianópolis → Porto Alegre (2)

EXERCÍCIO 3

- Um entregador de sucos precisa traçar a melhor rota entre lojas para entregar os pedidos rapidamente. Use Dijkstra para descobrir o menor tempo de entrega de sucos de Loja A até Loja F.
- Rotas:
 - $A \rightarrow B$ (1)
 - $A \rightarrow C$ (4)
 - $B \rightarrow D$ (2)
 - $C \rightarrow D$ (1)
 - $D \rightarrow E$ (3)
 - $E \rightarrow F$ (2)
- Desafios:
 - Liste os caminhos visitados em DFS e BFS a partir de A.
 - Qual a menor distância de A até F?

EXERCÍCIO 4

- Um mago precisa viajar por reinos para encontrar o pergaminho sagrado no Reino Z. O grafo representa portais mágicos com o custo de energia (peso) para usá-los.
- Portais mágicos:
 - $X \rightarrow Y$ (6)
 - $X \rightarrow W$ (2)
 - $W \rightarrow Y$ (2)
 - $Y \rightarrow Z$ (3)
 - $W \rightarrow Z$ (7)
- Objetivos:
 - Mostre a matriz de adjacência.
 - Calcule o caminho com menor custo de energia de X até Z.
 - Compare os caminhos encontrados em DFS e BFS a partir de X.

EXERCÍCIO 5

- Um personagem está em um labirinto com túneis de diferentes dificuldades (pesos). Ele precisa encontrar o caminho mais fácil até a saída.
- Túneis do labirinto:
 - Entrada $\rightarrow$ A (2)
 - A $\rightarrow$ B (2)
 - B $\rightarrow$ Saída (1)
 - Entrada $\rightarrow$ C (5)
 - C $\rightarrow$ Saída (1)
- Tarefas:
 - Modele esse labirinto como um grafo.
 - Use dijkstra('Entrada') para descobrir a melhor rota até 'Saída'.
 - Compare com os caminhos encontrados por DFS e BFS.

OBRIGADO!

- Encontre este **material on-line** em:
 - Slides: Plataforma Classroom
- Em caso de **dúvidas**, entre em contato:
 - **Prof. Tiago:** tiago.silva@cps.sp.gov.br

